Sí, es totalmente posible crear un formulario que liste, inserte, modifique y elimine registros directamente desde la misma pantalla, sin necesidad de escribir código PL/SQL manualmente. La clave está en aprovechar la funcionalidad nativa de Oracle Forms y sus asistentes.

🧙‍♂️ El Asistente de Bloque de Datos: Tu aliado sin código

La forma más rápida y limpia de lograr esto es utilizando el **Asistente de Bloque de Datos (Data Block Wizard)**. Este asistente no solo crea la estructura del bloque, sino que genera automáticamente los disparadores (triggers) necesarios para que las operaciones de inserción, actualización y borrado funcionen contra la base de datos.

Paso a paso:

1.En Forms Builder, ve a **Herramientas (Tools)** → **Asistente de Bloque de Datos (Data Block Wizard)**.
2.Selecciona **Tabla o Vista (Table or View)** y elige tu tabla `PUNTOS_VENTA`.
3.Selecciona las columnas que quieras mostrar en el listado.
4.Al finalizar, el asistente te preguntará si quieres crear el diseño (Layout). Acepta y usa el **Asistente de Diseño (Layout Wizard)** para elegir un estilo **Tabular** (para ver varios registros a la vez).

Al terminar, Forms habrá creado un bloque de datos con todos los disparadores internos que gestionan el DML (Data Manipulation Language) automáticamente. **No necesitas escribir una sola línea de PL/SQL** para que el CRUD básico funcione.

🛠️ La Barra de Herramientas (Toolbar) Predeterminada

Además del bloque, Oracle Forms incluye por defecto una **barra de herramientas** y un menú que ya traen los botones para realizar las operaciones CRUD.

•**Botón Guardar (Save):** Envía los cambios pendientes a la base de datos. Funciona tanto para registros nuevos (INSERT) como modificados (UPDATE).
•**Botón Eliminar (Delete):** Borra el registro seleccionado actualmente.
•**Botón Consultar (Query):** Permite entrar en modo de consulta para filtrar los registros que se muestran en el listado.

•**Botón Nuevo (New/Add):** Inserta una nueva fila vacía en el bloque para que el usuario ingrese los datos de un nuevo registro.

Esta barra de herramientas es **automática** y se activa por defecto en cada formulario. Puedes usarla tal cual sin necesidad de configurar nada.

⚠ ¿Cuándo sí necesitas programar en PL/SQL?

Aunque el CRUD básico es automático, hay situaciones en las que **sí** necesitarás escribir algo de código:

•**Validaciones personalizadas:** Si quieres que un campo solo acepte ciertos valores, o que se valide algo contra otra tabla antes de guardar.
•**Lógica de negocio compleja:** Por ejemplo, si al insertar un punto de venta también debes actualizar otra tabla o registrar un histórico.
•**Manejo de errores específicos:** Si quieres mostrar mensajes personalizados en lugar de los errores estándar de Oracle.
•**Personalización de la barra de herramientas:** Si quieres crear tu propia barra con botones específicos, o modificar el comportamiento de los botones estándar.

En resumen: **puedes crear un formulario CRUD completo sin escribir PL/SQL, usando los asistentes y la barra de herramientas estándar.** Esto te permite tener una pantalla funcional en muy poco tiempo. Solo necesitarás añadir código si requieres funcionalidades más allá de lo básico.

> El error que ves ocurre porque Oracle Forms, por defecto, trata el ítem del ID como un campo obligatorio y navegable. Al ser una columna generada automáticamente por la base de datos (identity o secuencia), el formulario no debería pedirte que la digites, pero necesita saber que no debe hacerlo.
>
> Para solucionarlo, debes ajustar las propiedades del ítem y, si es necesario, añadir un pequeño trigger. Aquí tienes los pasos exactos:

🎯 Paso 1: Ajustar las propiedades del ítem del ID

Selecciona el ítem correspondiente al ID (ej. `ID_PUNTO_VENTA`) en el Navegador de Objetos y ve a la **Paleta de Propiedades**. Cambia lo siguiente:

Propiedad	Valor	¿Por qué?
Requerido (Required)	**No**	Evita que Forms exija un valor antes de guardar.
Navegable (Navigable)	**No**	El usuario no podrá moverse a este campo ni editarlo.
Actualización Permitida (Update Allowed)	**No**	Evita que se modifique el ID en actualizaciones.
Inserción Permitida (Insert Allowed)	**No**	Evita que Forms intente insertar un valor nulo en el ID.

Con esto, el ítem se vuelve de solo lectura y el formulario no te pedirá que lo digites.

🔧 Paso 2: Si usas una SECUENCIA (no identity)

Si tu tabla usa una secuencia (no una columna identity), debes asignar el valor de la secuencia en el trigger `PRE-INSERT` a nivel de bloque. Este trigger se ejecuta justo antes de insertar el registro en la base de datos.

1. En el Navegador de Objetos, haz clic derecho sobre el **bloque de datos** (no sobre el ítem) y selecciona **Crear Disparador (Create Trigger)** → **PRE-INSERT**.
2. Escribe este código:

```plsql
BEGIN
   -- Asigna el siguiente valor de la secuencia al ítem del ID
   SELECT seq_puntos_venta.NEXTVAL
     INTO :BLK_PUNTOS_VENTA.ID_PUNTO_VENTA
     FROM DUAL;
```

```
END;
```

> **Nota:** Cambia `seq_puntos_venta` por el nombre real de tu secuencia y `BLK_PUNTOS_VENTA.ID_PUNTO_VENTA` por el nombre real de tu bloque e ítem.

🗄️ Paso 3: Si usas una columna IDENTITY

Si tu columna es `GENERATED ALWAYS AS IDENTITY` (Oracle 12c+), el formulario **no debe intentar insertar ningún valor** en esa columna. En este caso, la solución es la misma que en el Paso 1: el ítem debe ser **No Navegable** y **No Requerido**. El formulario simplemente omitirá esa columna en el `INSERT`, y la base de datos generará el valor automáticamente.

⚙️ Paso 4: Revisar la propiedad "Forzar Clave Primaria" del bloque

En algunos casos, si el bloque tiene la propiedad **"Forzar Clave Primaria" (Enforce Primary Key)** en **Sí**, Oracle Forms puede requerir que la clave primaria esté presente antes de guardar. Si tu tabla ya tiene la clave primaria definida y el ítem del ID está configurado como clave primaria, puedes dejar esta propiedad en **Sí**. Si te sigue dando problemas, cámbiala a **No** temporalmente para probar.

✅ Resumen de la solución

1. En el ítem del ID: **Requerido = No**, **Navegable = No**.
2. Si usas secuencia: añade el trigger `PRE-INSERT` para asignar `seq.NEXTVAL`.
3. Si usas identity: no hagas nada más, el formulario omitirá la columna automáticamente.

Con estos cambios, el formulario ya no te pedirá el ID y podrás guardar los registros sin problemas.

Entiendo perfectamente tu necesidad. Quieres un listado basado en una consulta personalizada (no una tabla simple) que ya trae un `WHERE` con un parámetro tipo `&punto&`, y además permitir filtros adicionales desde la pantalla. La clave está en **no usar el** `&` directamente en el bloque de datos (Oracle Forms no lo soporta en `Query Data Source Name`), sino manejar el filtro mediante `DEFAULT_WHERE` y un bloque de control. Aquí tienes la guía completa, con nombres en español, sin asistentes y con ejemplos probados.

🎯 Entendiendo el problema del `&` en Oracle Forms

En SQL*Plus, `&punto&` es una variable de sustitución que pide un valor al usuario. En Oracle Forms, ese mecanismo **no existe** para bloques de datos. La forma correcta es:

1. Crear un **campo de filtro** en un bloque de control (ej. `BLK_FILTRO.IT_PUNTO`).
2. Usar un trigger `PRE-QUERY` o un botón "Buscar" para construir dinámicamente la cláusula `WHERE` y asignarla con `SET_BLOCK_PROPERTY(..., DEFAULT_WHERE, ...)`.
3. Ejecutar `EXECUTE_QUERY` para que el bloque traiga solo los registros filtrados.

📋 Paso 1: Crear el bloque de datos manualmente

1. En el **Navegador de Objetos**, clic derecho sobre **Bloques de Datos** → **Crear**.
2. Elige **"Crear un nuevo bloque de datos manualmente"** → **Aceptar**.
3. Selecciona el bloque y en la **Paleta de Propiedades** configura:

Propiedad (Español)	Valor
Nombre	`BLK_PV`
Bloque de Datos de Base de Datos	`Sí`
Tipo de Origen de Datos de Consulta	`Consulta FROM` (FROM clause query)
Nombre del Origen de Datos de Consulta	`(SELECT pv.id_punto_venta, pv.nombre_pv, c.nombre_ciudad FROM tbpuntosventa pv JOIN ciudad c ON c.idciudad = pv.idciudad WHERE 1=1)`
Consultar Todos los Registros	`No`
Número de Registros Mostrados	`10`
Forzar Clave Primaria	`No`
Actualización Permitida	`No`
Inserción Permitida	`No`
Eliminación Permitida	`No`

> **Nota crucial:** La consulta debe ir **entre paréntesis**. Si no, recibirás error `FRM-41380`. El `WHERE 1=1` es un truco para que siempre puedas concatenar `AND ...` sin preocuparte por si ya hay un `WHERE`.

📦 Paso 2: Crear los ítems del bloque

Crea un ítem por cada columna que devuelve el `SELECT`. Los nombres **deben coincidir exactamente** con los alias del `SELECT`.

Nombre del Ítem	Tipo	Tipo de Dato	Lienzo (Canvas)
`ID_PUNTO_VENTA`	Text Item	Number	`CVS_LISTA`
`NOMBRE_PV`	Text Item	Char	`CVS_LISTA`

Nombre del Ítem	Tipo	Tipo de Dato	Lienzo (Canvas)
`NOMBRE_CIUDAD`	Text Item	Char	`CVS_LISTA`

Para cada uno: **Navegable = Sí**, **Requerido = No**.

🔍 Paso 3: Crear el bloque de filtros (Control Block)

1. Clic derecho sobre **Bloques de Datos** → **Crear** → **"Crear manualmente"**.
2. Property Palette:

- **Nombre:** `BLK_FILTRO`
- **Bloque de Datos de Base de Datos:** `No`

3. Crea los ítems:

Nombre	Tipo	Prompt	Ancho	Lienzo
`IT_PUNTO`	Text Item	`Nombre Punto:`	200	`CVS_LISTA`
`BT_BUSCAR`	Push Button	—	—	`CVS_LISTA`
`BT_LIMPIAR`	Push Button	—	—	`CVS_LISTA`

Asigna **Etiqueta** = `Buscar` al botón `BT_BUSCAR` y `Limpiar` al `BT_LIMPIAR`.

🧠 Paso 4: Trigger `PRE-QUERY` en `BLK_PV` (para filtros automáticos)

Este trigger se dispara **antes** de cualquier `EXECUTE_QUERY` y construye el `DEFAULT_WHERE` a partir del campo de filtro.

1. Clic derecho sobre el bloque `BLK_PV` → **Crear Disparador** → **PRE-QUERY**.
2. Código:

```plsql
```

```
DECLARE
    v_where VARCHAR2(4000) := '1=1';
BEGIN
    -- Si el usuario escribió algo en el filtro, agregarlo
    IF :BLK_FILTRO.IT_PUNTO IS NOT NULL THEN
        v_where := v_where || ' AND UPPER(nombre_pv) LIKE ''%'' ||
UPPER(:BLK_FILTRO.IT_PUNTO) || ''%''';
    END IF;

    -- Asignar el WHERE al bloque
    SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, v_where);
END;
```

> **¿Por qué `UPPER` y `LIKE`?** Para que la búsqueda no distinga mayúsculas/minúsculas y coincida con cualquier parte del nombre.

⭕ Paso 5: Trigger del botón `BT_BUSCAR`

En el bloque `BLK_FILTRO`, trigger `WHEN-BUTTON-PRESSED` de `BT_BUSCAR`:

plsql

```
BEGIN
    GO_BLOCK('BLK_PV');
    EXECUTE_QUERY;
END;
```

Explicación: Al presionar Buscar, el `PRE-QUERY` del bloque `BLK_PV` se dispara automáticamente y aplica el filtro antes de ejecutar la consulta.

🧹 Paso 6: Trigger del botón `BT_LIMPIAR`

plsql

```
BEGIN
    :BLK_FILTRO.IT_PUNTO := NULL;
```

```
    SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, '1=1');
    GO_BLOCK('BLK_PV');
    EXECUTE_QUERY;
END;
```

🚀 Paso 7: Cargar el listado al abrir el formulario

Crea un trigger `WHEN-NEW-FORM-INSTANCE` a nivel de módulo:

plsql

```
BEGIN
    :BLK_FILTRO.IT_PUNTO := NULL;
    GO_BLOCK('BLK_PV');
    EXECUTE_QUERY;
END;
```

📝 Ejemplo completo de la consulta en `Query Data Source Name`

sql

```
(SELECT pv.id_punto_venta,
         pv.nombre_pv,
         c.nombre_ciudad
    FROM tbpuntosventa pv
    JOIN ciudad c ON c.idciudad = pv.idciudad
  WHERE 1=1)
```

> **Importante:** No incluyas el `WHERE nombre like '&punto&'` original. Ese filtro se reemplaza por el `DEFAULT_WHERE` dinámico del `PRE-QUERY`. El `WHERE 1=1` queda como base para poder concatenar `AND` sin problemas.

⚠ Consideraciones sobre DML (Insertar/Actualizar/Eliminar)

Un bloque basado en `FROM clause query` **no permite DML** (insertar, actualizar, eliminar) directamente, porque Oracle Forms no sabe a qué tabla actualizar. Si necesitas CRUD, tienes dos opciones:

1. **Usar una vista** con `INSTEAD OF` triggers en la base de datos.
2. **Usar un bloque basado en tabla** (pero entonces no puedes usar la consulta con JOIN directamente).

Para tu caso (solo listado y filtros), el bloque `FROM clause query` es perfecto.

📌 Resumen de propiedades clave (en español)

Propiedad (Inglés)	Propiedad (Español)	Valor
Query Data Source Type	Tipo de Origen de Datos de Consulta	`Consulta FROM`
Query Data Source Name	Nombre del Origen de Datos de Consulta	`(SELECT ...)`
Query All Records	Consultar Todos los Registros	`No`
Number of Records Displayed	Número de Registros Mostrados	`10`
DEFAULT_WHERE	(se asigna por código)	`1=1` + filtros

Con esta guía, tu bloque mostrará el listado basado en la consulta con JOIN, y el usuario podrá escribir un nombre en el filtro y presionar Buscar para ver solo los registros que coincidan. Si algún trigger te da error, pégame el mensaje exacto y lo resolvemos.

Guía con comandos comentados para trabajar con Azure DevOps

A continuación tienes un script completo, con cada comando explicado. Está pensado para ejecutarse en **Git Bash** (o CMD/PowerShell en Windows, con las rutas adaptadas). La URL es ficticia.

bash

```bash
# ============================================================================
# SCRIPT GIT BASH PARA TRABAJAR CON AZURE DEVOPS
# URL ficticia de ejemplo:
# https://dev.azure.com/miempresa/CoreAcselX/_git/RepositorioFormularios
# ============================================================================

# ----------------------------------------------------------------------------
# PASO 0: CONFIGURACIÓN INICIAL (una sola vez por máquina)
# ----------------------------------------------------------------------------
# Identifica quién eres en los commits. Si no lo haces, Git se queja.
git config --global user.name "Tu Nombre Apellido"
git config --global user.email "tu.correo@miempresa.com"

# Verificar que quedó bien
git config --global --list

# ----------------------------------------------------------------------------
# PASO 1: CLONAR EL REPOSITORIO (primera vez en tu máquina)
# ----------------------------------------------------------------------------
# Ir a la carpeta donde quieres guardar el proyecto
cd /c/Users/tu_usuario/Documents/Proyectos

# Clonar el repositorio remoto. Azure te pedirá usuario + token.
git clone https://dev.azure.com/miempresa/CoreAcselX/_git/RepositorioFormularios

# Entrar a la carpeta del proyecto recién clonada
cd RepositorioFormularios

# Ver en qué rama estás (por defecto: main o master)
```

```
git branch

# Ver todas las ramas, incluidas las remotas
git branch -a

# ------------------------------------------------------------------------
# PASO 2: CAMBIAR A LA RAMA DE TRABAJO
# ------------------------------------------------------------------------
# Si ya existe la rama "desarrollo" en el remoto, cambiar a ella
git checkout desarrollo

# Si NO existe todavía, crear una nueva rama local y publicarla
# git checkout -b feat/mi-tarea          # Crea la rama
# git push -u origin feat/mi-tarea       # La sube al servidor por primera vez

# ------------------------------------------------------------------------
# PASO 3: TRAER LOS ÚLTIMOS CAMBIOS DEL SERVIDOR (antes de editar)
# ------------------------------------------------------------------------
# Descarga cambios SIN aplicarlos (solo consulta)
git fetch

# Descarga Y aplica los cambios a tu rama local (recomendado antes de editar)
git pull origin desarrollo

# Ver el estado actual del repositorio (qué archivos cambiaron)
git status

# ------------------------------------------------------------------------
# PASO 4: EDITAR ARCHIVOS
# ------------------------------------------------------------------------
# Aquí no hay comandos Git. Abres tu editor (Forms Builder, PL/SQL Developer,
# VS Code, Notepad++) y modificas los archivos que necesites.

# Ver qué archivos modificaste
git status

# Ver los cambios línea por línea
git diff

# Ver los cambios de un archivo específico
git diff forms/FOR_BLOQUEO_PV.fmb
```

```
# ------------------------------------------------------------------------
# PASO 5: PREPARAR LOS CAMBIOS (staging)
# ------------------------------------------------------------------------
# Agregar TODOS los cambios
git add .

# O agregar solo archivos específicos
git add forms/FOR_BLOQUEO_PV.fmb
git add scripts/pr_sinc_bloqueos.sql

# Ver qué quedó listo para commit (los verdes ya están en staging)
git status

# ------------------------------------------------------------------------
# PASO 6: CREAR EL COMMIT (guardar cambios localmente)
# ------------------------------------------------------------------------
# Un commit agrupa tus cambios con un mensaje descriptivo.
git commit -m "feat: agregar formulario de bloqueo de puntos de venta"

# Buenas prácticas de mensaje:
# feat:      nueva funcionalidad
# fix:       corrección de bug
# docs:      documentación
# refactor:  reorganización sin cambio funcional
# chore:     tareas varias

# ------------------------------------------------------------------------
# PASO 7: SUBIR LOS CAMBIOS AL SERVIDOR (push)
# ------------------------------------------------------------------------
# Subir al branch actual
git push origin desarrollo

# Si es la primera vez que subes un branch nuevo:
# git push -u origin feat/mi-tarea

# ------------------------------------------------------------------------
# PASO 8: MANEJO DE CONFLICTOS (merge)
# ------------------------------------------------------------------------
# Esto ocurre cuando otro compañero subió cambios al mismo archivo
# que tú editaste, y los cambios se pisaron.
```

```
# 1) Traer lo del servidor. Si hay conflicto, Git lo dirá.
git pull origin desarrollo

# 2) Ver qué archivos tienen conflicto
git status
# Los archivos conflictivos aparecen como "both modified"

# 3) Abrir el archivo conflictivo. Verás marcas así:
# <<<<<<< HEAD
#     -- Tu versión (local)
# SELECT * FROM punto_venta WHERE estado = 'ACTIVO';
# =======
#     -- Versión del servidor (compañero)
# SELECT * FROM punto_venta WHERE estado = 'INACTIVO';
# >>>>>>> origin/desarrollo

# 4) Editar el archivo y dejar SOLO la versión correcta
#    (borrar las líneas <<<<<<<, =======, >>>>>>>)
#    Si quieres conservar ambos cambios, escribe la versión combinada.

# 5) Marcar el archivo como resuelto
git add forms/FOR_BLOQUEO_PV.fmb
# (repetir para cada archivo en conflicto)

# 6) Terminar el merge con un commit
git commit -m "merge: resolver conflictos con desarrollo"

# 7) Subir la resolución al servidor
git push origin desarrollo

# ------------------------------------------------------------------------
# PASO 9: SI NO QUIERES HACER MERGE TODAVÍA (guardar cambios en el cajón)
# ------------------------------------------------------------------------
# Guarda temporalmente tus cambios sin hacer commit.
# Útil cuando necesitas traer lo del servidor pero aún no terminaste.
git stash

# Traer lo del servidor (ahora no habrá conflicto porque tus cambios
# están guardados temporalmente)
git pull origin desarrollo
```

```
# Recuperar tus cambios (puede haber conflicto aquí)
git stash pop

# Si hay conflicto, resolver con los pasos del Paso 8.

# ------------------------------------------------------------------------
# PASO 10: COMANDOS ÚTILES DEL DÍA A DÍA
# ------------------------------------------------------------------------

# Ver el historial de commits (últimos 10 en una línea)
git log --oneline -10

# Ver diferencias entre tu rama y el remoto
git fetch
git log HEAD..origin/desarrollo --oneline

# Descartar TODOS los cambios locales (¡CUIDADO! No se puede deshacer)
# git reset --hard HEAD

# Descartar cambios de un solo archivo
# git checkout -- forms/FOR_BLOQUEO_PV.fmb

# Deshacer el último commit pero mantener los cambios en staging
# git reset --soft HEAD~1

# Eliminar un branch local (después de fusionarlo)
# git branch -d feat/mi-tarea

# Eliminar un branch remoto
# git push origin --delete feat/mi-tarea

# Ver el remote configurado
git remote -v

# ------------------------------------------------------------------------
# PASO 11: FLUJO COMPLETO DEL DÍA A DÍA (resumen)
# ------------------------------------------------------------------------
# 1. cd /c/Users/tu_usuario/Documents/Proyectos/RepositorioFormularios
# 2. git checkout desarrollo
# 3. git pull origin desarrollo
```

```
# 4. Editar archivos con tu editor
# 5. git status
# 6. git add .
# 7. git commit -m "feat: descripción del cambio"
# 8. git push origin desarrollo
# 9. Si hay conflicto: git pull, resolver, git add, git commit, git push
```

🔑 Notas importantes sobre Azure DevOps

Autenticación con Token (PAT)

Azure ya no acepta la contraseña normal de tu cuenta. Debes crear un **Personal Access Token (PAT)**:

1.Ve a `https://dev.azure.com/miempresa/_usersSettings/tokens`
2.Clic en **"New Token"**
3.Nombre: `git-local`
4.Scopes: **Code (Read & Write)**
5.Copia el token (solo se muestra una vez)
6.Úsalo como **contraseña** cuando Git te lo pida

Para guardarlo permanentemente en Windows:

```bash
git config --global credential.helper manager
```

Archivo `.gitignore` (evitar subir archivos innecesarios)

Crea un archivo `.gitignore` en la raíz del repositorio con este contenido:

```text
# Compilados de Oracle Forms
*.fmx
*.mmx
*.plx
```

```
*.rep

# Temporales
*.tmp
*.log
*.bak

# IDE
.vscode/
.idea/

# Windows
Thumbs.db
```

Esto evita que subas archivos compilados que se regeneran al compilar.

Reglas de oro

1. **Nunca trabajes en `main` directamente.** Siempre en una rama propia.
2. **Siempre `git pull` antes de empezar a editar.**
3. **Commits pequeños y frecuentes** en lugar de uno gigante al final.
4. **Mensajes claros** en el commit.
5. **Nunca uses `git push --force`** en ramas compartidas.
6. **Si tienes dudas, `git status` te dice dónde estás.**

Con este script tienes todo lo necesario para conectarte, clonar, editar, subir y resolver conflictos con tu repositorio de Azure DevOps.